FINAL CODES

Matlab codes:

```
%% STEP 0 — Setup + clean (self-contained, no helpers)
clc; clearvars; close all; rng('default'); rng(42);

% --- INPUTS ---
workDir = fullfile("G:\D:\weedgraphnet\Nas Project\update imagery\Updated
clipped\Smallarea","small image matlab");
inRGB = fullfile(workDir,"..","small_valid_8bit.tif"); % RGB or RGB+NIR
inLBL = fullfile(workDir,"..","label_small.tif"); % uint8
{1=weed,2=rice,3=others,(0=unlabeled)}

% --- OUTPUTS ---
tilesDir = fullfile(workDir,"tiles");
imgOutDir = fullfile(tilesDir,"images");
lblOutDir = fullfile(tilesDir,"labels");
listDir = fullfile(workDir,"lists");
trainDir = fullfile(workDir,"train");
metricsDir= fullfile(workDir,"metrics");
cellfun(@(p)~exist(p,"dir")&&mkdir(p),
{tilesDir,imgOutDir,lblOutDir,listDir,trainDir,metricsDir});

% Clean manifests & previews safely (no external functions)
for f = ["train_manifest.txt","val_manifest.txt"]
fp = fullfile(listDir,f);
if exist(fp,"file"), delete(fp); end
end
prevDir = fullfile(metricsDir,"previews");
if exist(prevDir,"dir"), rmdir(prevDir,"s"); end
mkdir(prevDir);

% --- CLASSES & TRAIN PARAMS ---
classNames = ["others","rice","weed"]; % order matters
labelIDs = [3 2 1];
tileSize = [512 512];
tileStride = [512 512];
Ntrain = 400;
Nval = 20;

% --- Agri reasoning thresholds (used later) ---
veg.tauVARI = 0.05; % RGB veg threshold
veg.tauNDVI = 0.10; % NDVI threshold if 4th band exists
veg.minSat = 0.15; % low saturation => likely bare (OTHERS)
```

```
veg.grayD = 18; % |R-G| and |G-B| <= grayD ⇒ grayish

% --- Sanity: files exist ---
assert(isfile(inRGB), "RGB not found: %s", inRGB);
assert(isfile(inLBL), "Label not found: %s", inLBL);

% --- Quick info (inline band count; no helper) ---
infoI = imfinfo(inRGB); H = infoI(1).Height; W = infoI(1).Width;
infoL = imfinfo(inLBL); HL = infoL(1).Height; WL = infoL(1).Width;
assert(H==HL && W==WL, "Image/Label size mismatch.");

% Determine number of bands robustly
if numel(infoI)==1
if isfield(infoI,'BitsPerSample') && ~isempty(infoI.BitsPerSample)
nb = numel(infoI.BitsPerSample);
else
nb = 3; % fallback
end
else
if isfield(infoI(1),'BitsPerSample') && ~isempty(infoI(1).BitsPerSample)
nb = numel(infoI(1).BitsPerSample);
else
nb = 3;
end
end

fprintf('Using RGB : %s (%dx%dx~%d)\n', inRGB, H, W, nb);
fprintf('Using LBL : %s (%dx%d, 1 band)\n\n', inLBL, HL, WL);

disp("STEP 0 ready. Proceed to STEP 1.");

%%

%% STEP 1 — Tile image+label into disk (no helpers)
clc; rng('default'); rng(42);

% --- Must match STEP 0 paths/vars ---
workDir = fullfile("G:\D:\weedgraphnet\Nas Project\update imagery\Updated
clipped\Smallarea","small image matlab");
inRGB = fullfile(workDir,"..","small_valid_8bit.tif");
inLBL = fullfile(workDir,"..","label_small.tif");

tilesDir = fullfile(workDir,"tiles");
imgOutDir = fullfile(tilesDir,"images");
```

```
lblOutDir = fullfile(tilesDir,"labels");
if ~exist(imgOutDir,"dir"), mkdir(imgOutDir); end
if ~exist(lblOutDir,"dir"), mkdir(lblOutDir); end

tileSize = [512 512];
tileStride = [512 512];

% --- Quick info (consistent with STEP 0) ---
infoI = imfinfo(inRGB); H = infoI(1).Height; W = infoI(1).Width;
infoL = imfinfo(inLBL); HL = infoL(1).Height; WL = infoL(1).Width;
assert(H==HL && W==WL, "Image/Label size mismatch.");

% robust band count
if isscalar(infoI)
nb = (isfield(infoI,'BitsPerSample') && ~isempty(infoI.BitsPerSample)) *
numel(infoI.BitsPerSample);
if nb==0, nb = 3; end
else
if isfield(infoI(1),'BitsPerSample') && ~isempty(infoI(1).BitsPerSample)
nb = numel(infoI(1).BitsPerSample);
else
nb = 3;
end
end
useBands = 1:min(3,nb); % use RGB only (first 3 bands)

fprintf('Image %dx%dx%d | Labels %dx%d\n', W,H,nb, WL,HL);

% --- Compute tile grid ---
ts = tileSize; st = tileStride;
rows = 1:st(1):H; cols = 1:st(2):W;
if rows(end) ~= H - ts(1) + 1, rows = [rows, max(1, H-ts(1)+1)]; end
if cols(end) ~= W - ts(2) + 1, cols = [cols, max(1, W-ts(2)+1)]; end

% --- Tiling loop ---
counter = 0;
for r = rows
r1 = r; r2 = min(H, r1 + ts(1) - 1);
for c = cols
c1 = c; c2 = min(W, c1 + ts(2) - 1);

% Read sub-image (crop) — use PixelRegion when available
try
% Read all bands then select the first 3
```

```
Icrop = imread(inRGB, 'PixelRegion', {[r1 r2], [c1 c2]});
if ndims(Icrop)==2, Icrop = repmat(Icrop,[1 1 3]); end
if size(Icrop,3) >= 3
Icrop = Icrop(:,:,useBands);
else
% pad to 3 channels if needed
I3 = zeros(size(Icrop,1), size(Icrop,2), 3, 'like', Icrop);
I3(:,:,1:size(Icrop,3)) = Icrop;
Icrop = I3;
end
catch
% fallback: read then crop (slower but robust)
I = imread(inRGB);
I = I(:,:,useBands);
Icrop = I(r1:r2, c1:c2, :);
end

% Read label crop
try
Lcrop = imread(inLBL, 'PixelRegion', {[r1 r2], [c1 c2]});
catch
L = imread(inLBL);
Lcrop = L(r1:r2, c1:c2);
end
Lcrop = uint8(Lcrop); % 0=unlabeled, 1=weed, 2=rice, 3=others

% If edge tiles are smaller, pad to exact tile size (reflect)
if any(size(Icrop,1:2) ~= ts)
Ipad = padarray(Icrop, ts - size(Icrop,1:2), 'replicate', 'post');
Lpad = padarray(Lcrop, ts - size(Lcrop,1:2), 'replicate', 'post');
Icrop = Ipad; Lcrop = Lpad;
end

% Enforce allowed labels {0,1,2,3}
Lcrop(~ismember(Lcrop, uint8([0 1 2 3]))) = 0;

% Write files
counter = counter + 1;
imgName = sprintf('img_%05d.tif', counter);
lblName = sprintf('lbl_%05d.png', counter);
imwrite(Icrop, fullfile(imgOutDir, imgName), 'Compression','none');
imwrite(Lcrop, fullfile(lblOutDir, lblName));
end
end
```

```
fprintf('✓ Tiled %d patches → %s | %s\n', counter, imgOutDir, lblOutDir);
disp('STEP 1 done. Proceed to STEP 2.');

%%

%% =========================
% STEP 2 — Build TRAIN/VAL with guaranteed 3 classes (fast + robust)
% =========================
fprintf('[STEP 2-TURBO] Scanning & building manifests…\n');

% --- Paths (MUST match your Step 1 output) ---
workDir = fullfile("G:\D:\weedgraphnet\Nas Project\update imagery\Updated
clipped\Smallarea","small image matlab");
tilesDir = fullfile(workDir,"tiles");
imgOutDir = fullfile(tilesDir,"images");
lblOutDir = fullfile(tilesDir,"labels");
listDir = fullfile(workDir,"lists");
if ~exist(listDir,"dir"), mkdir(listDir); end
trainList = fullfile(listDir,'train_manifest.txt');
valList = fullfile(listDir,'val_manifest.txt');

% --- Enumerate tiles (sorted) ---
imgFiles = dir(fullfile(imgOutDir,"*.tif"));
lblFiles = dir(fullfile(lblOutDir,"*.png"));
assert(numel(imgFiles)==numel(lblFiles),'Tile count mismatch tiles\images vs
tiles\labels');
imgNames = string({imgFiles.name}); [~,si] = sort(imgNames); imgNames =
imgNames(si);
lblNames = string({lblFiles.name}); [~,sl] = sort(lblNames); lblNames =
lblNames(sl);
nTiles = numel(lblNames);

% --- Cache per-tile stats to avoid re-reading next time ---
cachePath = fullfile(listDir,"label_stats.mat");
needScan = true;
if isfile(cachePath)
S = load(cachePath);
if isfield(S,'pW') && numel(S.pW)==nTiles && isfield(S,'pR') && isfield(S,'pO')
&& isfield(S,'pureO')
pW=S.pW; pR=S.pR; pO=S.pO; pureO=S.pureO;
needScan = false;
fprintf(' Reusing cached per-tile stats → %s\n', cachePath);
end
```

```
end

if needScan
fprintf(' Scanning %d label tiles…\n', nTiles);
pW = zeros(nTiles,1,'single'); % frac weed (==1)
pR = zeros(nTiles,1,'single'); % frac rice (==2)
pO = zeros(nTiles,1,'single'); % frac others (==3)
pureO= false(nTiles,1); % all pixels == 3
t0 = tic;
for i = 1:nTiles
L = imread(fullfile(lblOutDir, lblNames(i)));
np = numel(L);
pW(i) = nnz(L==1)/np;
pR(i) = nnz(L==2)/np;
pO(i) = nnz(L==3)/np;
pureO(i) = all(L(:)==3);
if mod(i,200)==0 || i==nTiles
fprintf(' scanned %d/%d…\n', i, nTiles);
end
end
fprintf(' Scan done in %.2f s.\n', toc(t0));
save(cachePath,'pW','pR','pO','pureO','imgNames','lblNames');
fprintf(' Cached → %s\n', cachePath);
end

% --- Build pools with relaxed thresholds if needed ---
thrSeq = [0.01 0.005 0.001];
W_pool = []; R_pool = []; O_pool = [];
for pass = 1:numel(thrSeq)
pWmin = thrSeq(pass); pOmin = thrSeq(pass);
candW = find(pW >= pWmin);
candR = find(pR > 0);
candO = find(pO >= pOmin | pureO); % allow pure-O or mixed with some O
W_pool = candW(:); R_pool = candR(:); O_pool = candO(:);
fprintf(' Pass %d: thresholds pW>=%.4f, pO>=%.4f → W=%d R=%d O=%d\n', ...
pass, pWmin, pOmin, numel(W_pool), numel(R_pool), numel(O_pool));
if numel(W_pool)>=35 && numel(O_pool)>=30, break; end
end

fprintf('=== POOLS (final) === W=%d R=%d O=%d\n', numel(W_pool), numel(R_pool),
numel(O_pool));
assert(~isempty(W_pool) && ~isempty(R_pool) && ~isempty(O_pool), 'One of the
class pools is empty — check labels.');
```

```
% --- Targets ---
Ntrain = 400; Nval = 20;

% Validation target composition (soft): try 6/12/2 (W/R/O), clamp by
availability
valW = min(6, numel(W_pool));
valR = min(12, numel(R_pool));
valO = min(2, numel(O_pool));

% If short, top-up from largest remaining pools
short = Nval - (valW+valR+valO);
if short>0
remW = numel(W_pool)-valW; remR = numel(R_pool)-valR; remO =
numel(O_pool)-valO;
rems = [remR, remW, remO]; % <-- store, then index
[~,ord] = sort(rems,'descend'); % prefer R then W then O
add = [0 0 0];
for kk = 1:numel(ord)
k = ord(kk);
canAdd = max(0, rems(k));
addNow = min(short, canAdd);
add(k) = add(k) + addNow;
short = short - addNow;
if short<=0, break; end
end
% map add back to classes: [R W O]
valR = valR + add(1);
valW = valW + add(2);
valO = valO + add(3);
end
% Final clamp
valW = min(valW, numel(W_pool)); valR = min(valR, numel(R_pool)); valO =
min(valO, numel(O_pool));

% --- Sample VAL without replacement ---
rng('default'); rng(42);
permW = randperm(numel(W_pool), valW); VAL_W = W_pool(permW);
permR = randperm(numel(R_pool), valR); VAL_R = R_pool(permR);
permO = randperm(numel(O_pool), valO); VAL_O = O_pool(permO);
VAL = unique([VAL_W; VAL_R; VAL_O],'stable');

% --- Remove VAL indices from pools for TRAIN sampling ---
W_tr = setdiff(W_pool, VAL, 'stable');
R_tr = setdiff(R_pool, VAL, 'stable');
```

```
O_tr = setdiff(O_pool, VAL, 'stable');

% TRAIN desired class shares (soft). You can tweak.
tW = round(0.24 * Ntrain);
tR = round(0.56 * Ntrain);
tO = Ntrain - tW - tR;

pickNR = @(pool,n) pool( 1:min(n, numel(pool)) ); % no replacement
pickRep= @(pool,n) pool( randi(max(1,numel(pool)), n, 1) ); % with replacement

TW = [ pickNR(W_tr, tW); pickRep(W_tr, max(0,tW - numel(W_tr))) ];
TR = [ pickNR(R_tr, tR); pickRep(R_tr, max(0,tR - numel(R_tr))) ];
TO = [ pickNR(O_tr, tO); pickRep(O_tr, max(0,tO - numel(O_tr))) ];

TRAIN = unique([TW; TR; TO],'stable');

% --- Enforce exact VAL size (pad or trim) ---
if numel(VAL) > Nval
VAL = VAL(randperm(numel(VAL), Nval));
elseif numel(VAL) < Nval
ALLpad = unique([W_pool; R_pool; O_pool],'stable');
need = Nval - numel(VAL);
addIdx = ALLpad(randi(numel(ALLpad), need, 1));
VAL = [VAL; addIdx];
end
VAL = VAL(randperm(numel(VAL)));

% --- Enforce exact TRAIN size (pad or trim) ---
if numel(TRAIN) > Ntrain
TRAIN = TRAIN(randperm(numel(TRAIN), Ntrain));
elseif numel(TRAIN) < Ntrain
ALL_for_train = unique([W_tr; R_tr; O_tr],'stable');
if isempty(ALL_for_train), ALL_for_train = unique([W_pool; R_pool;
O_pool],'stable'); end
need = Ntrain - numel(TRAIN);
addIdx = ALL_for_train(randi(numel(ALL_for_train), need, 1));
TRAIN = [TRAIN; addIdx];
end
TRAIN = TRAIN(randperm(numel(TRAIN)));

% --- Report composition by pool membership ---
isIn = @(idx,set) nnz(ismember(idx,set));
fprintf('\n=== MANIFEST COMPOSITION ===\n');
```

```
fprintf('TRAIN %d → W=%d R=%d O=%d\n', numel(TRAIN), isIn(TRAIN,W_pool),
isIn(TRAIN,R_pool), isIn(TRAIN,O_pool));
fprintf('VAL %d → W=%d R=%d O=%d\n', numel(VAL), isIn(VAL,W_pool),
isIn(VAL,R_pool), isIn(VAL,O_pool));

% --- Write manifest files (img|label per line, no quotes, no blanks) ---
trainLines = string( fullfile(imgOutDir, imgNames(TRAIN)) ) + "|" +
string( fullfile(lblOutDir, lblNames(TRAIN)) );
valLines = string( fullfile(imgOutDir, imgNames(VAL)) ) + "|" +
string( fullfile(lblOutDir, lblNames(VAL)) );
writelines(trainLines, trainList);
writelines(valLines, valList);
fprintf('✓ Sampler saved → %s\n', trainList);
fprintf('✓ Sampler saved → %s\n', valList);

% --- Quick pixel-fraction sanity check on a subset of TRAIN ---
K = min(150, numel(TRAIN));
sub = TRAIN(1:K);
wFrac=0; rFrac=0; oFrac=0;
for k = 1:numel(sub)
L = imread(fullfile(lblOutDir, lblNames(sub(k))));
np = numel(L);
wFrac = wFrac + nnz(L==1)/np;
rFrac = rFrac + nnz(L==2)/np;
oFrac = oFrac + nnz(L==3)/np;
end
wFrac = 100*wFrac/K; rFrac = 100*rFrac/K; oFrac = 100*oFrac/K;
fprintf('Approx class fractions (first %d TRAIN tiles): W=%.2f%% R=%.2f%%
O=%.2f%%\n', K, wFrac, rFrac, oFrac);


%%

%% =========================
% STEP 3 — Train U-Net (with early stopping + checkpoints)
% =========================
fprintf('[STEP 3] Training U-Net…\n');

% --- Paths ---
workDir = fullfile("G:\D:\weedgraphnet\Nas Project\update imagery\Updated
clipped\Smallarea","small image matlab");
tilesDir = fullfile(workDir,"tiles");
imgDir = fullfile(tilesDir,"images");
lblDir = fullfile(tilesDir,"labels");
```

```
listDir = fullfile(workDir,"lists");
trainList = fullfile(listDir,'train_manifest.txt');
valList = fullfile(listDir,'val_manifest.txt');

% --- Read manifests robustly ---
trPairs = read_manifest_pairs(trainList);
vaPairs = read_manifest_pairs(valList);
assert(~isempty(trPairs) && ~isempty(vaPairs), 'Empty manifest after read.');

% Column vectors -> row vectors (pixelLabelDatastore prefers row)
trImgs = string(trPairs(:,1)).'; trLbls = string(trPairs(:,2)).';
vaImgs = string(vaPairs(:,1)).'; vaLbls = string(vaPairs(:,2)).';

% --- Datastores (first 3 bands if 4-band) ---
imdsTr = imageDatastore(cellstr(trImgs), 'ReadFcn', @(p)
im2uint8(readFirst3(p)));
pxdsTr = pixelLabelDatastore(cellstr(trLbls), ["others","rice","weed"], [3 2
1]);

imdsVa = imageDatastore(cellstr(vaImgs), 'ReadFcn', @(p)
im2uint8(readFirst3(p)));
pxdsVa = pixelLabelDatastore(cellstr(vaLbls), ["others","rice","weed"], [3 2
1]);

dsTr = combine(imdsTr, pxdsTr);
dsVa = combine(imdsVa, pxdsVa);

fprintf('Train pairs: %d | Val pairs: %d\n', numel(trImgs), numel(vaImgs));

% --- Estimate class weights (balances R dominance) ---
sampleN = min(80, numel(trLbls));
clsCnt = zeros(3,1); % [others rice weed]
for i = 1:sampleN
L = readLabel(trLbls{i}); % categorical
clsCnt(1) = clsCnt(1) + nnz(L=="others");
clsCnt(2) = clsCnt(2) + nnz(L=="rice");
clsCnt(3) = clsCnt(3) + nnz(L=="weed");
end
pxTotal = sum(clsCnt); pxFrac = max(clsCnt,1) / max(pxTotal,1);
wts = 1./pxFrac; wts = wts / max(wts);
wts = min(max(wts,0.25),5.0);
% extra boost for the hard classes
wts([1 3]) = wts([1 3]) * 1.5; % others & weed
```

```
fprintf('Class weights (others,rice,weed): [%.2f %.2f %.2f]\n', wts(1), wts(2),
wts(3));

% --- Network ---
inputSize = [512 512 3];
numClasses = 3;
% NOTE: unetLayers shows a deprecation warning in newer releases.
lgraph = unetLayers(inputSize, numClasses, 'EncoderDepth', 4);
lgraph = replaceLayer(lgraph, 'Segmentation-Layer', ...
pixelClassificationLayer('Name','Segmentation-Layer', ...
'Classes', ["others","rice","weed"], ...
'ClassWeights', wts));

% --- Training options (early stopping + checkpoints) ---
ckptDir = fullfile(workDir,'checkpoints');
if ~exist(ckptDir,'dir'), mkdir(ckptDir); end

% A reasonable validation frequency for your dataset size:
miniBatch = 4;
valFreq = 10; % validate every 10 iters (keeps checkpoints frequent)
maxEpochs = 8; % allow a few epochs; patience will stop it earlier if plateau

opts = trainingOptions('adam', ...
'InitialLearnRate',1e-3, ...
'MaxEpochs', maxEpochs, ...
'MiniBatchSize', miniBatch, ...
'Shuffle','every-epoch', ...
'ValidationData', dsVa, ...
'ValidationFrequency', valFreq, ...
'ValidationPatience', 2, ... % <-- patience: stop if val loss not improving
'L2Regularization', 1e-4, ...
'GradientThresholdMethod','l2norm', ...
'ExecutionEnvironment','auto', ... % GPU if available
'CheckpointPath', ckptDir, ... % <-- save snapshots
'Verbose', true, ...
'Plots','none');

% --- Train ---
net = trainNetwork(dsTr, lgraph, opts);

% --- Save final model package (plus class meta) ---
modelDir = fullfile(workDir,'train'); if ~exist(modelDir,'dir'),
mkdir(modelDir); end
stamp = string(datetime('now','Format','yyyyMMdd_HHmmss'));
```

```
modelPath = fullfile(modelDir, "model_unet_"+stamp+".mat");
classNames = ["others","rice","weed"]; labelIDs = [3 2 1]; inSize = inputSize;
save(modelPath, 'net','classNames','labelIDs','inSize','wts','-v7.3');
fprintf('✓ Model saved: %s\n', modelPath);

%% ===== Inline helpers (no external dependencies) =====
function pairs = read_manifest_pairs(pth)
txt = string(splitlines(string(fileread(pth))));
txt = strip(txt);
txt = txt(txt~="");
txt = replace(txt, """", "");
hasDelim = contains(txt,"|");
txt = txt(hasDelim);
if isempty(txt), pairs = strings(0,2); return; end
% fast split
C = cellfun(@(s) strsplit(s,'|'), cellstr(txt), 'uni', 0);
C = C(cellfun(@(c) numel(c)==2, C));
pairs = string(vertcat(C{:}));
end

function I3 = readFirst3(p)
I = imread(p);
if size(I,3) >= 3
I3 = I(:,:,1:3);
else
I3 = repmat(I(:,:,1),1,1,3);
end
end

function L = readLabel(p)
Li = imread(p);
if iscategorical(Li)
L = Li;
else
% Map 1=weed, 2=rice, 3=others → categorical
L = categorical(Li, [3 2 1], ["others","rice","weed"]);
end
end



%%
```

```
%% =========================
% STEP 4B — Recompute metrics + Confusion Matrix (val set)
% =========================
fprintf('[STEP 4B] Recomputing metrics + confusion matrix…\n');

% --- Paths (match earlier steps) ---
workDir = fullfile("G:\D:\weedgraphnet\Nas Project\update imagery\Updated
clipped\Smallarea","small image matlab");
tilesDir = fullfile(workDir,"tiles"); %#ok<NASGU>
listDir = fullfile(workDir,"lists");
valList = fullfile(listDir,'val_manifest.txt');
trainDir = fullfile(workDir,"train");
metDir = fullfile(workDir,"metrics");
if ~exist(metDir,"dir"), mkdir(metDir); end

% --- Load latest trained model package ---
mats = dir(fullfile(trainDir,"model_unet_*.mat"));
assert(~isempty(mats), 'No model package found in %s', trainDir);
[~,ix] = sort([mats.datenum],'descend');
modelPath = fullfile(trainDir, mats(ix(1)).name);
S = load(modelPath);
assert(isfield(S,'net'), 'Model file lacks variable ''net'': %s', modelPath);
net = S.net;

% class meta (robust defaults if not saved)
if isfield(S,'classNames'), classNames = string(S.classNames(:).'); else,
classNames = ["others","rice","weed"]; end
if isfield(S,'labelIDs'), labelIDs = double(S.labelIDs(:).'); else, labelIDs
= [3 2 1]; end
if isfield(S,'inSize'), inSize = S.inSize; else, inSize =
net.Layers(1).InputSize; end
K = numel(classNames);
fprintf(' Using model: %s\n', modelPath);
fprintf(' Classes: %s\n', strjoin(classNames,", "));

% --- Read validation manifest ---
pairs = read_manifest_pairs_S4B(valList); % [N x 2] strings
```

```
assert(~isempty(pairs), 'Validation manifest is empty: %s', valList);
N = size(pairs,1);
fprintf(' * Found %d validation pairs.\n', N);

% --- Accumulators ---
TP = zeros(1,K); FP = zeros(1,K); FN = zeros(1,K);
CM = zeros(K,K); % rows = GT, cols = PR
totLabPix = 0;

for i = 1:N
imgPath = strtrim(pairs(i,1));
lblPath = strtrim(pairs(i,2));

% Image (first 3 bands, uint8)
I = readFirst3_S4B(imgPath);
if ~isequal(size(I,1), inSize(1)) || ~isequal(size(I,2), inSize(2))
I = imresize(I, inSize(1:2), 'bilinear');
end

% Label -> categorical, then resize to match input
Li = imread(lblPath);
GT = categorical(Li, labelIDs, classNames, 'Ordinal',false);
if ~isequal(size(GT,1), inSize(1)) || ~isequal(size(GT,2), inSize(2))
GT = imresize(GT, inSize(1:2), 'nearest');
end

% Inference
PR = semanticseg(I, net, 'ExecutionEnvironment','auto');

% Mask to labeled pixels (ignore <undefined>)
M = false(size(GT));
for k=1:K, M = M | (GT==classNames(k)); end
if ~any(M(:)), continue; end

% Per-class PR/GT for PRF1/IoU
for k = 1:K
g = (GT==classNames(k)) & M;
p = (PR==classNames(k)) & M;
TP(k) = TP(k) + nnz(g & p);
FP(k) = FP(k) + nnz(~g & p);
FN(k) = FN(k) + nnz(g & ~p);
end

% Confusion matrix update
```

```
gtIdx = cat2idx_S4B(GT(M), classNames);
prIdx = cat2idx_S4B(PR(M), classNames);
CM = CM + accumarray([gtIdx, prIdx], 1, [K K]);
totLabPix = totLabPix + numel(gtIdx);

if mod(i,5)==0 || i==N
fprintf(' processed %d/%d\n', i, N);
end
end

% --- Per-class metrics ---
prec = TP ./ max(TP+FP, 1);
rec = TP ./ max(TP+FN, 1);
f1 = (2*prec.*rec) ./ max(prec+rec, 1e-12);
iou = TP ./ max(TP+FP+FN, 1);

% --- Overall metrics ---
overallAcc = sum(diag(CM)) / max(sum(CM,'all'),1);
macroF1 = mean(f1);
microTP = sum(TP); microFP = sum(FP); microFN = sum(FN);
microPrec = microTP / max(microTP+microFP,1);
microRec = microTP / max(microTP+microFN,1);
microF1 = (2*microPrec*microRec) / max(microPrec+microRec, 1e-12);
meanIoU = mean(iou);

fprintf('\n=== VALIDATION (STEP 4B) ===\n');
fprintf('Overall accuracy : %.4f\n', overallAcc);
fprintf('Macro F1 : %.4f\n', macroF1);
fprintf('Micro F1 : %.4f\n', microF1);
fprintf('Mean IoU : %.4f\n\n', meanIoU);

for k=1:K
fprintf('%-8s IoU=%.4f Prec=%.4f Rec=%.4f F1=%.4f (TP=%d FP=%d FN=%d)\n', ...
char(classNames(k)), iou(k), prec(k), rec(k), f1(k), TP(k), FP(k), FN(k));
end

% --- Save per-class table & confusion matrix ---
T = table(classNames(:), iou(:), prec(:), rec(:), f1(:), ...
'VariableNames', {'Class','IoU','Precision','Recall','F1'});
writetable(T, fullfile(metDir,'metrics_val_step4b.csv'));

% Confusion matrix CSV with headers
cmCSV = fullfile(metDir,'confusion_matrix_val.csv');
fid = fopen(cmCSV,'w'); assert(fid>0);
```

```
fprintf(fid,'GT\\PR,%s\n', strjoin(string(classNames),','));
for r=1:K
row = strjoin(string(CM(r,:)), ',');
fprintf(fid,'%s,%s\n', string(classNames(r)), row);
end
fclose(fid);

% Heatmap PNG (no GUI needed)
f = figure('Visible','off','Position',[100 100 700 560]);
imagesc(CM); axis image; colormap(parula); colorbar;
title(sprintf('Confusion Matrix (Val) — Acc=%.3f mIoU=%.3f', overallAcc,
meanIoU));
xticks(1:K); yticks(1:K);
xticklabels(cellstr(classNames)); yticklabels(cellstr(classNames));
xlabel('Predicted'); ylabel('Ground Truth');
textStrings = string(CM);
[x,y] = meshgrid(1:K,1:K);
for ii=1:numel(x)
text(x(ii), y(ii), textStrings(ii), 'HorizontalAlignment','center',
'Color','w', 'FontSize',10, 'FontWeight','bold');
end
exportgraphics(f, fullfile(metDir,'confusion_matrix_val.png'), 'Resolution',
200);
close(f);

fprintf('✓ Saved per-class metrics → %s\n',
fullfile(metDir,'metrics_val_step4b.csv'));
fprintf('✓ Saved confusion matrix → %s and PNG in %s\n', cmCSV, metDir);

%% ===== Helpers (STEP-4B only) =====
function pairs = read_manifest_pairs_S4B(pth)
txt = string(splitlines(string(fileread(pth))));
txt = strip(txt); txt = txt(txt~=""); txt = replace(txt, """", "");
txt = txt(contains(txt,"|"));
if isempty(txt), pairs = strings(0,2); return; end
C = cellfun(@(s) strsplit(s,'|'), cellstr(txt), 'uni', 0);
C = C(cellfun(@(c) numel(c)==2, C));
pairs = string(vertcat(C{:}));
end

function I3 = readFirst3_S4B(p)
I = imread(p);
if size(I,3) >= 3, I3 = I(:,:,1:3);
else, I3 = repmat(I(:,:,1),1,1,3);
```

```
end
I3 = im2uint8(I3);
end

function idx = cat2idx_S4B(C, classNames)
% Map categorical labels (limited to classNames) to indices 1..K
K = numel(classNames);
idx = zeros(numel(C),1);
for k=1:K
idx(C==classNames(k)) = k;
end
idx = max(idx,1); % safety
end



%%









%% =========================
% STEP 4D — Label-space TTA + Veg-gate + Morphology (FAST, no logits)
% =========================
fprintf('[STEP 4D] Label-space TTA + gating + morphology on validation…\n');

% --- Paths (match earlier steps) ---
workDir = fullfile("G:\D:\weedgraphnet\Nas Project\update imagery\Updated
clipped\Smallarea","small image matlab");
tilesDir = fullfile(workDir,"tiles");
imgDir = fullfile(tilesDir,"images"); %#ok<NASGU>
lblDir = fullfile(tilesDir,"labels"); %#ok<NASGU>
listDir = fullfile(workDir,"lists");
valList = fullfile(listDir,'val_manifest.txt');

metDir = fullfile(workDir,"metrics");
prevDir = fullfile(metDir,"previews_4D");
if ~exist(metDir,"dir"), mkdir(metDir); end
if exist(prevDir,"dir"), rmdir(prevDir,'s'); end, mkdir(prevDir);
```

```
% --- Load latest trained model package ---
trainDir = fullfile(workDir,"train");
mats = dir(fullfile(trainDir,"model_unet_*.mat"));
assert(~isempty(mats), 'No model package found in %s', trainDir);
[~,ix] = sort([mats.datenum],'descend');
modelPath = fullfile(trainDir, mats(ix(1)).name);
S = load(modelPath); % contains net (+ meta)
assert(isfield(S,'net'), 'Model file lacks variable ''net'': %s', modelPath);
net = S.net;

% class meta (robust defaults if not saved)
if isfield(S,'classNames'), classNames = string(S.classNames(:).'); else,
classNames = ["others","rice","weed"]; end
if isfield(S,'labelIDs'), labelIDs = double(S.labelIDs(:).'); else, labelIDs
= [3 2 1]; end
if isfield(S,'inSize'), inSize = S.inSize; else, inSize =
net.Layers(1).InputSize; end
K = numel(classNames);
fprintf(' Using model: %s\n', modelPath);

% --- Read validation manifest ---
pairs = read_manifest_pairs_S4D(valList); % [N x 2] strings: image | label
assert(~isempty(pairs), 'Validation manifest is empty or unreadable: %s',
valList);
N = size(pairs,1);
fprintf(' * Found %d validation pairs.\n', N);

% --- TTA & postproc knobs (gentle defaults) ---
USE_TTA = true; % 4-view voting
USE_VARI_GATE = true; % low-veg → others
VARI_THR = 0.05; % try 0.04–0.07 if needed
AREA_OPEN_PIX = 120; % remove tiny islands per class (0 = off)
MODEFILT_W = 0; % odd window for mode filter in index space, e.g., 3 or 5; 0
= off
MAX_PREV = 12; % preview panels

% --- Colormap for visualization (uint8 RGB) ---
C_others = uint8([150 150 150]);
C_rice = uint8([220 20 60]);
C_weed = uint8([255 215 0]);
cmap = uint8([C_weed; C_rice; C_others]); % row1=weed, row2=rice, row3=others
(for labelIDs mapping)

% --- Accumulators for metrics ---
```

```
TP = zeros(1,K); FP = zeros(1,K); FN = zeros(1,K);
CM = zeros(K,K);
savedPanels = 0;

for i = 1:N
imgPath = strtrim(pairs(i,1));
lblPath = strtrim(pairs(i,2));

% Load & prep image
I = imread(imgPath);
if size(I,3)<3, I = repmat(I(:,:,1),1,1,3); else, I = I(:,:,1:3); end
I = im2uint8(I);

% Resize to net input if needed
if ~isequal(size(I,1), inSize(1)) || ~isequal(size(I,2), inSize(2))
Irs = imresize(I, inSize(1:2), 'bilinear');
else
Irs = I;
end

% Ground truth categorical (resize to inSize)
Li = imread(lblPath);
GT = categorical(Li, labelIDs, classNames, 'Ordinal',false);
if ~isequal(size(GT,1), inSize(1)) || ~isequal(size(GT,2), inSize(2))
GT = imresize(GT, inSize(1:2), 'nearest');
end

% ===== 1) Label-space TTA predictions =====
if USE_TTA
PR0 = semanticseg(Irs, net, 'ExecutionEnvironment','auto'); % orig
PR1 = semanticseg(fliplr(Irs), net, 'ExecutionEnvironment','auto'); PR1 =
fliplr(PR1); % LR
PR2 = semanticseg(flipud(Irs), net, 'ExecutionEnvironment','auto'); PR2 =
flipud(PR2); % UD
PR3 = semanticseg(flipud(fliplr(Irs)), net, 'ExecutionEnvironment','auto');
PR3 = fliplr(flipud(PR3)); % LR+UD
PR = vote4_S4D(PR0, PR1, PR2, PR3, classNames); % categorical
else
PR = semanticseg(Irs, net, 'ExecutionEnvironment','auto');
end

% ===== 2) Optional VARI veg gate (label-space) =====
if USE_VARI_GATE
VARI = vari_from_rgb_S4D(Irs);
```

```
lowVeg = VARI < VARI_THR;
% Only push weed/rice to others at low veg
PR(lowVeg & (PR=="weed" | PR=="rice")) = "others";
end

% ===== 3) Morphology: area-open per class & optional mode filter =====
% Work in index space 1..K for morphology
prIdx = cat2idx2D_S4D(PR, classNames); % uint8 1..K
if AREA_OPEN_PIX > 0
for kk = 1:K
bw = (prIdx == kk);
bw = bwareaopen(bw, AREA_OPEN_PIX);
% pixels removed default to others (class "others")
if any(~bw & prIdx==kk,'all')
othersK = find(classNames=="others"); if isempty(othersK), othersK=1; end
prIdx((prIdx==kk) & ~bw) = uint8(othersK);
end
end
end
if MODEFILT_W >= 3 && mod(MODEFILT_W,2)==1
prIdx = modefilt2_S4D(prIdx, MODEFILT_W);
end
% Back to categorical PR after morphology
PR = idx2cat2D_S4D(prIdx, classNames);

% ===== 4) Metrics (size-safe) =====
% Convert GT to index map
gtIdx = cat2idx2D_S4D(GT, classNames);
[gtIdx, prIdx] = ensure_same_size_idx_S4D(gtIdx, prIdx);
M = gtIdx > 0; % labeled mask

for kk = 1:K
TP(kk) = TP(kk) + nnz( (gtIdx==kk) & (prIdx==kk) );
FP(kk) = FP(kk) + nnz( (gtIdx~=kk) & (prIdx==kk) & M );
FN(kk) = FN(kk) + nnz( (gtIdx==kk) & (prIdx~=kk) );
end
CM = CM + accumarray([double(gtIdx(M)), double(prIdx(M))], 1, [K K]);

% ===== 5) Save a few previews =====
if savedPanels < MAX_PREV
% Build nice 2x2 panel
GTn = labelIndexFromCategorical_S4D(GT, classNames, labelIDs); % uint8 in
{labelIDs}
PRn = labelIndexFromCategorical_S4D(PR, classNames, labelIDs);
```

```
GTrgb = labelRGB_S4D(GTn, cmap);
PRrgb = labelRGB_S4D(PRn, cmap);
overlay = blend_S4D(Irs, PRrgb, 0.45);
panel = makePanel_S4D(Irs, GTrgb, PRrgb, overlay, ["Image","Ground
truth","Prediction","Overlay"]);
[~, base, ~] = fileparts(imgPath);
outPng = fullfile(prevDir, sprintf('val4D_%03d_%s.png', i, base));
imwrite(panel, outPng);
savedPanels = savedPanels + 1;
end

if mod(i,5)==0 || i==N
fprintf(' * Processed %d/%d validation tiles…\n', i, N);
end
end

% --- Summaries ---
prec = TP ./ max(TP+FP,1);
rec = TP ./ max(TP+FN,1);
f1 = (2*prec.*rec) ./ max(prec+rec, 1e-12);
iou = TP ./ max(TP+FP+FN,1);
macroF1 = mean(f1);
microF1 = sum(TP) / max(sum(TP)+sum(FP),1); % = overall acc for
one-label-per-pixel
meanIoU = mean(iou);
overall = sum(TP) / max(sum(TP)+sum(FP),1);

fprintf('\n=== VALIDATION (STEP 4D) ===\n');
fprintf('Overall accuracy : %.4f\n', overall);
fprintf('Macro F1 : %.4f\n', macroF1);
fprintf('Micro F1 : %.4f\n', microF1);
fprintf('Mean IoU : %.4f\n\n', meanIoU);
for k=1:K
fprintf('%-8s IoU=%.4f Prec=%.4f Rec=%.4f F1=%.4f (TP=%d FP=%d FN=%d)\n', ...
char(classNames(k)), iou(k), prec(k), rec(k), f1(k), TP(k), FP(k), FN(k));
end

% --- Save metrics & confusion ---
csvPath = fullfile(metDir, 'metrics_val_step4d.csv');
T = table(classNames(:), iou(:), prec(:), rec(:), f1(:), ...
'VariableNames', {'Class','IoU','Precision','Recall','F1'});
writetable(T, csvPath);

cmPathCsv = fullfile(metDir, 'confusion_matrix_val_4d.csv');
```

```
fid = fopen(cmPathCsv,'w');
fprintf(fid,'GT\\PRED');
for k=1:K, fprintf(fid,',%s', char(classNames(k))); end
fprintf(fid,"\n");
for r=1:K
fprintf(fid, '%s', char(classNames(r)));
for c=1:K
fprintf(fid, ',%d', CM(r,c));
end
fprintf(fid, "\n");
end
fclose(fid);

fprintf('✓ Saved metrics → %s\n', csvPath);
fprintf('✓ Saved confusion → %s\n', cmPathCsv);
fprintf('✓ Wrote %d previews → %s\n', savedPanels, prevDir);

%% ===== STEP 4D helpers =====
function pairs = read_manifest_pairs_S4D(pth)
txt = string(splitlines(string(fileread(pth))));
txt = strip(txt); txt = txt(txt~=""); txt = replace(txt, """", "");
txt = txt(contains(txt,"|"));
if isempty(txt), pairs = strings(0,2); return; end
C = cellfun(@(s) strsplit(s,'|'), cellstr(txt), 'uni', 0);
C = C(cellfun(@(c) numel(c)==2, C));
pairs = string(vertcat(C{:}));
end

function VARI = vari_from_rgb_S4D(Iu8)
I = im2double(Iu8);
R = I(:,:,1); G = I(:,:,2); B = I(:,:,3);
VARI = (G - R) ./ max(G + R - B, 1e-6);
end

function PR = vote4_S4D(A,B,C,D, classNames)
% Majority vote among 4 categorical maps; ties broken by preferring the mode
of (A,B)
sz = size(A);
K = numel(classNames);
% Convert to index maps 1..K
Ia = cat2idx2D_S4D(A, classNames);
Ib = cat2idx2D_S4D(B, classNames);
Ic = cat2idx2D_S4D(C, classNames);
Id = cat2idx2D_S4D(D, classNames);
```

```
stack = cat(3, Ia, Ib, Ic, Id); % HxWx4
% mode across 3rd dim
PRidx = mode(stack, 3);
% In rare 4-way ties (almost impossible here), fall back to A
PRidx(PRidx==0) = Ia(PRidx==0);
PR = idx2cat2D_S4D(PRidx, classNames);
end

function idx2D = cat2idx2D_S4D(C, classNames)
% Map categorical HxW to class indices 1..K (0 for undefined)
K = numel(classNames);
idx2D = zeros(size(C), 'uint8');
for k = 1:K
idx2D(C == classNames(k)) = k;
end
end

function C = idx2cat2D_S4D(idx, classNames)
K = numel(classNames); %#ok<NASGU>
C = categorical(zeros(size(idx)), 1:numel(classNames), cellstr(classNames));
for k = 1:numel(classNames)
C(idx==k) = classNames(k);
end
end

function [A,B] = ensure_same_size_idx_S4D(A,B)
sa = size(A); sb = size(B);
if numel(sa)>2, sa=sa(1:2); end
if numel(sb)>2, sb=sb(1:2); end
H=min(sa(1),sb(1)); W=min(sa(2),sb(2));
A = A(1:H,1:W); B = B(1:H,1:W);
end

function idx = labelIndexFromCategorical_S4D(C, classNames, labelIDs)
% Return uint8 of labelIDs (not 1..K), used for coloring
idx = zeros(size(C), 'uint8');
for k = 1:numel(classNames)
idx(C == classNames(k)) = uint8(labelIDs(k));
end
end

function RGB = labelRGB_S4D(Lu, cmap)
% cmap rows: [weed; rice; others]; labels are 1,2,3 accordingly
LUT = zeros(256,3,'uint8');
```

```
LUT(1,:) = cmap(1,:); % 1 -> weed
LUT(2,:) = cmap(2,:); % 2 -> rice
LUT(3,:) = cmap(3,:); % 3 -> others
RGB = ind2rgb_uint8_S4D(Lu, LUT);
end

function J = blend_S4D(I, M, alpha)
I = im2double(I); M = im2double(M);
J = im2uint8((1-alpha)*I + alpha*M);
end

function panel = makePanel_S4D(I, GTrgb, PRrgb, overlay, titles)
tileH = size(I,1); tileW = size(I,2);
pad = 10; cellW = tileW; cellH = tileH + 30;
W = 2*cellW + 3*pad; H = 2*cellH + 3*pad;
panel = uint8(255*ones(H, W, 3));
cells = {I, GTrgb; PRrgb, overlay};
for r = 1:2
for c = 1:2
y0 = pad + (r-1)*(cellH+pad) + 30;
x0 = pad + (c-1)*(cellW+pad);
panel(y0:y0+tileH-1, x0:x0+tileW-1, :) = cells{r,c};
try
tpos = [x0+5, y0-25];
panel = insertText(panel, tpos, titles((r-1)*2+c), 'FontSize', 18, ...
'BoxColor','white','BoxOpacity',0.0,'TextColor','black');
catch
end
end
end
end

function RGB = ind2rgb_uint8_S4D(L, LUT)
sz = size(L);
L = uint16(L)+1; % safety shift
R = reshape(LUT(L,1), sz);
G = reshape(LUT(L,2), sz);
B = reshape(LUT(L,3), sz);
RGB = cat(3, R, G, B);
end
```

```
%%
```

```
%% =========================
% STEP 4E_5 — Pick tiles for accuracy + figure w/ legend
% Uses STEP 4D v1 post-processing (TTA + VARI-gate + morph open).
% Outputs: PNG/PDF figure + CSV; adds color legend (weed/rice/others).
% =========================
fprintf('[STEP 4E_5] Searching for VERY CLEAN, HIGH-ACCURACY tiles…\n');

% --- Paths ---
workDir = fullfile("G:\D:\weedgraphnet\Nas Project\update imagery\Updated
clipped\Smallarea","small image matlab");
listDir = fullfile(workDir,"lists");
valList = fullfile(listDir,'val_manifest.txt');

metDir = fullfile(workDir,"metrics");
if ~exist(metDir,"dir"), mkdir(metDir); end
outPng = fullfile(metDir, "figure_val_top3_clean_v1.png");
outPdf = fullfile(metDir, "figure_val_top3_clean_v1.pdf");
outCsv = fullfile(metDir, "figure_val_top3_clean_v1_metrics.csv");

% --- Load latest model package ---
trainDir = fullfile(workDir,"train");
mats = dir(fullfile(trainDir,"model_unet_*.mat"));
assert(~isempty(mats), 'No model package found in %s', trainDir);
[~,ix] = sort([mats.datenum],'descend');
modelPath = fullfile(trainDir, mats(ix(1)).name);
S = load(modelPath); assert(isfield(S,'net')); net = S.net;

% --- Meta ---
if isfield(S,'classNames'), classNames = string(S.classNames(:).'); else,
classNames = ["others","rice","weed"]; end
if isfield(S,'labelIDs'), labelIDs = double(S.labelIDs(:).'); else, labelIDs
= [3 2 1]; end
```

```
if isfield(S,'inSize'), inSize = S.inSize; else, inSize =
net.Layers(1).InputSize; end
K = numel(classNames);
kOthers = find(classNames=="others"); if isempty(kOthers), kOthers=1; end
kRice = find(classNames=="rice"); if isempty(kRice), kRice=2; end
kWeed = find(classNames=="weed"); if isempty(kWeed), kWeed=3; end

% --- Read validation manifest ---
pairs = read_manifest_pairs_S4E5(valList);
assert(~isempty(pairs), 'Validation manifest is empty or unreadable: %s',
valList);
N = size(pairs,1);
fprintf(' Using model: %s\n', modelPath);
fprintf(' * Found %d validation pairs.\n', N);

% --- STEP 4D (v1) post-proc knobs (from your best run) ---
USE_TTA = true; % 4-view vote
USE_VARI_GATE = true; % low-veg -> others
VARI_THR = 0.05;
AREA_OPEN_PIX = 120;
MODEFILT_W = 0; % off (as v1)

% --- Strictness & cleanliness thresholds (tighter first, then relax) ---
F1_targets = [0.90 0.85 0.80 0.75];
IoU_targets = [0.80 0.75 0.70 0.65];
GT_MIN_SHARE = 0.002; % each class present (>=0.2% GT)
MAX_ISLANDS = [12 18 24 30]; % max small components per class
MAX_BOUNDARY_NO = [0.080 0.110 0.140 0.180]; % boundary pixels / total

% --- Score all tiles ---
tiles = struct('idx',{},'iou',{},'f1',{},'prec',{},'rec',{}, ...
'macroF1',{},'meanIoU',{},'gtShare',{}, ...
'islands',{},'boundaryNoise',{},'Irs',{},'GTn',{},'PRn',{});
fprintf(' * Scoring tiles (metrics + cleanliness)…\n');

for i = 1:N
imgPath = strtrim(pairs(i,1));
lblPath = strtrim(pairs(i,2));

% Image
I = imread(imgPath); if size(I,3)<3, I=repmat(I(:,:,1),1,1,3); else,
I=I(:,:,1:3); end
I = im2uint8(I);
if size(I,1)~=inSize(1) || size(I,2)~=inSize(2)
```

```
Irs = imresize(I, inSize(1:2), 'bilinear');
else
Irs = I;
end

% GT
Li = imread(lblPath);
GT = categorical(Li, labelIDs, classNames, 'Ordinal', false);
if size(GT,1)~=inSize(1) || size(GT,2)~=inSize(2)
GT = imresize(GT, inSize(1:2), 'nearest');
end
gtIdx = cat2idx2D_S4E5(GT, classNames);

% Predict with STEP 4D v1 post-proc
if USE_TTA
PR0 = semanticseg(Irs, net, 'ExecutionEnvironment','auto');
PR1 = fliplr(semanticseg(fliplr(Irs), net, 'ExecutionEnvironment','auto'));
PR2 = flipud(semanticseg(flipud(Irs), net, 'ExecutionEnvironment','auto'));
PR3 = fliplr(flipud(semanticseg(flipud(fliplr(Irs)), net,
'ExecutionEnvironment','auto')));
PR = vote4_S4E5(PR0,PR1,PR2,PR3,classNames);
else
PR = semanticseg(Irs, net, 'ExecutionEnvironment','auto');
end

if USE_VARI_GATE
VARI = vari_from_rgb_S4E5(Irs);
lowVeg = VARI < VARI_THR;
PR(lowVeg & (PR=="weed" | PR=="rice")) = "others";
end

prIdx = cat2idx2D_S4E5(PR, classNames);
if AREA_OPEN_PIX>0
for kk=1:K
bw=(prIdx==kk); bw = bwareaopen(bw, AREA_OPEN_PIX);
prIdx((prIdx==kk) & ~bw) = uint8(kOthers);
end
end
if MODEFILT_W>=3 && mod(MODEFILT_W,2)==1
prIdx = modefilt2_S4E5(prIdx, MODEFILT_W);
end

[gtIdx, prIdx] = ensure_same_size_idx_S4E5(gtIdx, prIdx);
M = gtIdx>0;
```

```
% Metrics
iou=zeros(1,K); prec=iou; rec=iou; f1=iou; gtShare=iou;
for kk=1:K
tp=nnz((gtIdx==kk)&(prIdx==kk));
fp=nnz((gtIdx~=kk)&(prIdx==kk)&M);
fn=nnz((gtIdx==kk)&(prIdx~=kk));
denI=tp+fp+fn;
iou(kk) = (denI>0) * double(tp)/max(denI,1);
pden=tp+fp; rden=tp+fn;
prec(kk)= (pden>0) * double(tp)/max(pden,1);
rec(kk) = (rden>0) * double(tp)/max(rden,1);
d=prec(kk)+rec(kk);
f1(kk) = (d>0) * (2*prec(kk)*rec(kk)/max(d,eps));
gtShare(kk)= nnz((gtIdx==kk)&M)/max(nnz(M),1);
end

% Cleanliness: small-island count and boundary-noise
islands = zeros(1,K); bNoise = zeros(1,K);
for kk=1:K
bw = (prIdx==kk);
CC = bwconncomp(bw);
smallCount = 0;
for c=1:CC.NumObjects
if numel(CC.PixelIdxList{c}) <= 64, smallCount = smallCount + 1; end
end
islands(kk) = smallCount;
per = bwperim(bw);
bNoise(kk) = nnz(per)/numel(bw);
end

tiles(i).idx=i;
tiles(i).iou=iou; tiles(i).f1=f1; tiles(i).prec=prec; tiles(i).rec=rec;
tiles(i).macroF1=mean(f1); tiles(i).meanIoU=mean(iou);
tiles(i).gtShare=gtShare;
tiles(i).islands=islands; tiles(i).boundaryNoise=bNoise;
tiles(i).Irs = Irs;
tiles(i).GTn = mapLabel_S4E5(gtIdx, classNames, labelIDs);
tiles(i).PRn = mapLabel_S4E5(prIdx, classNames, labelIDs);
end

% --- Strict -> relaxed search for very-clean tiles ---
picked = [];
picked_thr = [NaN NaN NaN NaN];
```

```
for t = 1:numel(F1_targets)
fthr = F1_targets(t); iothr = IoU_targets(t);
maxIsl = MAX_ISLANDS(t); maxBN = MAX_BOUNDARY_NO(t);

ok = false(1,N);
for i=1:N
ts = tiles(i);
if all(ts.gtShare > GT_MIN_SHARE) && ...
all(ts.f1 >= fthr) && all(ts.iou >= iothr) && ...
all(ts.islands <= maxIsl) && ...
all(ts.boundaryNoise <= maxBN)
ok(i) = true;
end
end
cand = find(ok);
if ~isempty(cand)
% Rank by minF1 -> MacroF1 -> mIoU, then cleanliness
sc = zeros(numel(cand), 5);
for j=1:numel(cand)
ts = tiles(cand(j));
sc(j,1) = min(ts.f1);
sc(j,2) = ts.macroF1;
sc(j,3) = ts.meanIoU;
sc(j,4) = -sum(ts.islands);
sc(j,5) = -sum(ts.boundaryNoise);
end
[~,ord] = sortrows(sc, [-1 -2 -3 -4 -5]);
picked = cand(ord(1:min(3,numel(cand))));
picked_thr = [fthr, iothr, maxIsl, maxBN];
fprintf(' * Found %d clean tiles at F1>=%.2f IoU>=%.2f (islands<=%d,
bNoise<=%.3f): pick %s\n', ...
numel(cand), fthr, iothr, maxIsl, maxBN, mat2str(picked));
break
end
end

% Fallback if none pass any tier: pick best available by composite score
if isempty(picked)
fprintf(' ! No tiles met strict clean+accuracy tiers; selecting best available
by composite score.\n');
allIdx = 1:N;
SC = zeros(N,5);
for i=1:N
ts = tiles(i);
```

```
SC(i,1)= min(ts.f1);
SC(i,2)= ts.macroF1;
SC(i,3)= ts.meanIoU;
SC(i,4)= -sum(ts.islands);
SC(i,5)= -sum(ts.boundaryNoise);
end
[~,ord] = sortrows(SC, [-1 -2 -3 -4 -5]);
picked = allIdx(ord(1:min(3,N)));
end

fprintf(' * Final picked indices: %s\n', mat2str(picked));

% --- Colors + legend swatches ---
C_others = uint8([150 150 150]);
C_rice = uint8([220 20 60]);
C_weed = uint8([255 215 0]);
cmap = uint8([C_weed; C_rice; C_others]); % [weed; rice; others]

% --- Compose figure (add a legend row at bottom) ---
nRows = numel(picked) + 1; % extra row for legend
nCols = 4;
fig = figure('Color','w','Units','pixels','Position',[70 70 2600
max(1,520*nRows)]); %#ok<LFIG>
tlo = tiledlayout(nRows, nCols, 'TileSpacing','compact','Padding','compact');
colTitles = ["Image","Ground truth","Segmentation (V1)","Overlay"];

CSVrows = {};
for rr = 1:numel(picked)
ii = picked(rr);
Irs = tiles(ii).Irs;
GTn = tiles(ii).GTn;
PRn = tiles(ii).PRn;

GTrgb = labelRGB_S4E5(GTn, cmap);
PRrgb = labelRGB_S4E5(PRn, cmap);
OL = blend_S4E5(Irs, PRrgb, 0.42);

panels = {Irs, GTrgb, PRrgb, OL};
for cc=1:nCols
ax = nexttile(tlo, (rr-1)*nCols + cc);
imshow(panels{cc}, 'Parent', ax);
if rr==1, title(ax, colTitles(cc), 'FontWeight','bold','FontSize',14); end
axis(ax,'image','off');
end
```

```
% Per-row caption text with numbers (rightmost tile)
t = tiles(ii);
axR = nexttile(tlo, (rr-1)*nCols + nCols);
hold(axR,'on');
txt1 = sprintf('F1 (others/rice/weed) = %.3f / %.3f / %.3f', t.f1(kOthers),
t.f1(kRice), t.f1(kWeed));
txt2 = sprintf('IoU (others/rice/weed) = %.3f / %.3f / %.3f', t.iou(kOthers),
t.iou(kRice), t.iou(kWeed));
txt3 = sprintf('Macro-F1 = %.3f | mIoU = %.3f', t.macroF1, t.meanIoU);
txt4 = sprintf('Islands (o/r/w) = %d / %d / %d | bNoise (o/r/w) = %.3f / %.3f
/ %.3f', ...
t.islands(kOthers), t.islands(kRice), t.islands(kWeed), ...
t.boundaryNoise(kOthers), t.boundaryNoise(kRice), t.boundaryNoise(kWeed));
annotation('textbox', [axR.Position(1)+axR.Position(3)-0.32,
axR.Position(2)+0.01, 0.32, 0.19], ...
'String', {txt1, txt2, txt3, txt4}, 'FitBoxToText','on',
'EdgeColor','none', ...
'Color',[0 0 0], 'FontSize',10, 'HorizontalAlignment','right');
hold(axR,'off');

% CSV
CSVrows(end+1,:) = {ii, pairs(ii,1), pairs(ii,2), ...
t.f1(kOthers), t.f1(kRice), t.f1(kWeed), ...
t.iou(kOthers), t.iou(kRice), t.iou(kWeed), ...
t.macroF1, t.meanIoU, ...
t.islands(kOthers), t.islands(kRice), t.islands(kWeed), ...
t.boundaryNoise(kOthers), t.boundaryNoise(kRice),
t.boundaryNoise(kWeed)}; %#ok<AGROW>
end

% --- Legend row (bottom) ---
legRow = nRows;
axL = nexttile(tlo, (legRow-1)*nCols + 1, [1 nCols]); % span across columns
axis(axL,'off'); hold(axL,'on');

swW = 80; shW = 28; fs=13;
x0 = 80; y0 = 40;
% Weed swatch
rectangle('Position',[x0 y0 swW shW], 'FaceColor', double(C_weed)/255,
'EdgeColor','none', 'Parent', axL);
text(x0+swW+10, y0+shW-6, 'Weed', 'FontWeight','bold', 'FontSize', fs,
'Parent', axL);
% Rice swatch
```

```
x1 = x0 + swW + 220;
rectangle('Position',[x1 y0 swW shW], 'FaceColor', double(C_rice)/255,
'EdgeColor','none', 'Parent', axL);
text(x1+swW+10, y0+shW-6, 'Rice', 'FontWeight','bold', 'FontSize', fs,
'Parent', axL);
% Others swatch
x2 = x1 + swW + 220;
rectangle('Position',[x2 y0 swW shW], 'FaceColor', double(C_others)/255,
'EdgeColor','none', 'Parent', axL);
text(x2+swW+10, y0+shW-6, 'Others', 'FontWeight','bold', 'FontSize', fs,
'Parent', axL);

text(x2+swW+230, y0+shW-6, 'Legend (class colors)', 'FontAngle','italic',
'FontSize', fs, 'Parent', axL);

hold(axL,'off');

% --- Save outputs ---
drawnow;
exportgraphics(fig, outPng, 'Resolution', 600);
exportgraphics(fig, outPdf, 'ContentType','vector');
fprintf('✓ Saved figure →\n PNG: %s\n PDF: %s\n', outPng, outPdf);

T = cell2table(CSVrows, 'VariableNames', { ...
'val_index','image_path','label_path', ...
'F1_others','F1_rice','F1_weed', ...
'IoU_others','IoU_rice','IoU_weed', ...
'MacroF1','MeanIoU', ...
'Islands_others','Islands_rice','Islands_weed', ...
'bNoise_others','bNoise_rice','bNoise_weed'});
writetable(T, outCsv);
fprintf('✓ Saved per-tile CSV → %s\n', outCsv);
if ~isnan(picked_thr(1))
fprintf('✓ Threshold tier used: F1≥%.2f, IoU≥%.2f, islands≤%d,
bNoise≤%.3f\n', ...
picked_thr(1), picked_thr(2), picked_thr(3), picked_thr(4));
end

%% ===== Helpers (_S4E5) =====
function pairs = read_manifest_pairs_S4E5(pth)
txt = string(splitlines(string(fileread(pth))));
txt = strip(txt); txt = txt(txt~=""); txt = replace(txt, """", ""); txt =
txt(contains(txt,"|"));
if isempty(txt), pairs = strings(0,2); return; end
```

```
C = cellfun(@(s) strsplit(s,'|'), cellstr(txt), 'uni', 0); C = C(cellfun(@(c)
numel(c)==2, C));
pairs = string(vertcat(C{:}));
end

function VARI = vari_from_rgb_S4E5(Iu8)
I = im2double(Iu8); R=I(:,:,1); G=I(:,:,2); B=I(:,:,3);
VARI = (G - R) ./ max(G + R - B, 1e-6);
end

function PR = vote4_S4E5(A,B,C,D, classNames)
Ia = cat2idx2D_S4E5(A, classNames); Ib = cat2idx2D_S4E5(B, classNames);
Ic = cat2idx2D_S4E5(C, classNames); Id = cat2idx2D_S4E5(D, classNames);
stack = cat(3, Ia, Ib, Ic, Id); PRidx = mode(stack, 3);
PR = idx2cat2D_S4E5(PRidx, classNames);
end

function idx2D = cat2idx2D_S4E5(C, classNames)
K=numel(classNames); idx2D=zeros(size(C),'uint8');
for k=1:K, idx2D(C==classNames(k))=k; end
end

function C = idx2cat2D_S4E5(idx, classNames)
C = categorical(zeros(size(idx)), 1:numel(classNames), cellstr(classNames));
for k=1:numel(classNames), C(idx==k)=classNames(k); end
end

function [A,B] = ensure_same_size_idx_S4E5(A,B)
sa=size(A); sb=size(B); if numel(sa)>2, sa=sa(1:2); end; if numel(sb)>2,
sb=sb(1:2); end
H=min(sa(1),sb(1)); W=min(sa(2),sb(2)); A=A(1:H,1:W); B=B(1:H,1:W);
end

function L = mapLabel_S4E5(idx, classNames, labelIDs)
L = zeros(size(idx), 'uint8');
for k=1:numel(classNames)
L(idx==k) = uint8(labelIDs(k));
end
end

function RGB = labelRGB_S4E5(Lu, cmap)
LUT = zeros(256,3,'uint8'); LUT(1,:)=cmap(1,:); LUT(2,:)=cmap(2,:);
LUT(3,:)=cmap(3,:);
sz=size(Lu); Lu16=uint16(Lu)+1;
```

```
R=reshape(LUT(Lu16,1),sz); G=reshape(LUT(Lu16,2),sz);
B=reshape(LUT(Lu16,3),sz);
RGB=cat(3,R,G,B);
end

function J = blend_S4E5(I, M, alpha)
I=im2double(I); M=im2double(M); J=im2uint8((1-alpha)*I + alpha*M);
end

function out = modefilt2_S4E5(idx, w)
if ~(w>=3 && mod(w,2)==1), out=idx; return; end
pad=floor(w/2); idxp=padarray(idx,[pad pad],'replicate','both'); out=idx;
for r=1:size(idx,1)
for c=1:size(idx,2)
win=idxp(r:r+2*pad, c:c+2*pad); out(r,c)=mode(win(:));
end
end
end
```

```
%%
```

```
%% =========================
% STEP 4F — Journal-style accuracy table (overall + per-class)
% Fixed LaTeX writer (no string/logical arithmetic in formatter)
% Outputs:
% - metrics_table_integrated.csv
% - metrics_table_integrated.tex
% =========================
fprintf('[STEP 4F] Building journal-style accuracy table (overall +
per-class)…\n');

% --- Paths & meta ---
workDir = fullfile("G:\D:\weedgraphnet\Nas Project\update imagery\Updated
clipped\Smallarea","small image matlab");
listDir = fullfile(workDir,"lists");
```

```
valList = fullfile(listDir,'val_manifest.txt');

metDir = fullfile(workDir,"metrics");
if ~exist(metDir,"dir"), mkdir(metDir); end
outCsv = fullfile(metDir, "metrics_table_integrated.csv");
outTex = fullfile(metDir, "metrics_table_integrated.tex");

% --- Load latest trained model package ---
trainDir = fullfile(workDir,"train");
mats = dir(fullfile(trainDir,"model_unet_*.mat"));
assert(~isempty(mats), 'No model package found in %s', trainDir);
[~,ix] = sort([mats.datenum],'descend');
modelPath = fullfile(trainDir, mats(ix(1)).name);
S = load(modelPath); assert(isfield(S,'net'), 'Model lacks ''net'': %s',
modelPath);
net = S.net;

% --- Class meta ---
if isfield(S,'classNames'), classNames = string(S.classNames(:).'); else,
classNames = ["others","rice","weed"]; end
if isfield(S,'labelIDs'), labelIDs = double(S.labelIDs(:).'); else, labelIDs
= [3 2 1]; end
if isfield(S,'inSize'), inSize = S.inSize; else, inSize =
net.Layers(1).InputSize; end
K = numel(classNames);
kOthers = find(classNames=="others"); if isempty(kOthers), kOthers=1; end
kRice = find(classNames=="rice"); if isempty(kRice), kRice=2; end
kWeed = find(classNames=="weed"); if isempty(kWeed), kWeed=3; end

% --- Read validation manifest ---
pairs = read_manifest_pairs_TAB(valList);
assert(~isempty(pairs), 'Validation manifest empty: %s', valList);
N = size(pairs,1);
fprintf(' Using model: %s\n', modelPath);
fprintf(' * Found %d validation pairs.\n', N);

% --- STEP 4D v1 post-proc knobs ---
USE_TTA = true; % 4-view voting in label space
USE_VARI_GATE = true; % vegetation gate
VARI_THR = 0.05;
AREA_OPEN_PIX = 120; % remove tiny islands per class
MODEFILT_W = 0; % off

% --- Accumulators ---
```

```
TP = zeros(1,K); FP = zeros(1,K); FN = zeros(1,K); TN = zeros(1,K);
classPixelsGT = zeros(1,K);
confMat = zeros(K,K); % rows: GT, cols: PR
totalCorrect = 0; totalPixels = 0;

% --- Loop over validation tiles ---
for i = 1:N
imPath = strtrim(pairs(i,1));
lbPath = strtrim(pairs(i,2));

% Image (first 3 bands)
I = imread(imPath);
if size(I,3) < 3, I = repmat(I(:,:,1),1,1,3); else, I = I(:,:,1:3); end
I = im2uint8(I);
if size(I,1)~=inSize(1) || size(I,2)~=inSize(2)
Irs = imresize(I, inSize(1:2), 'bilinear');
else
Irs = I;
end

% Ground truth -> categorical -> index map {1..K}
Li = imread(lbPath);
GT = categorical(Li, labelIDs, classNames, 'Ordinal', false);
if size(GT,1)~=inSize(1) || size(GT,2)~=inSize(2)
GT = imresize(GT, inSize(1:2), 'nearest');
end
gtIdx = cat2idx2D_TAB(GT, classNames); % 1..K
maskL = gtIdx > 0;

% Predict with label-space TTA
if USE_TTA
PR0 = semanticseg(Irs, net, 'ExecutionEnvironment','auto');
PR1 = fliplr(semanticseg(fliplr(Irs), net, 'ExecutionEnvironment','auto'));
PR2 = flipud(semanticseg(flipud(Irs), net, 'ExecutionEnvironment','auto'));
PR3 = fliplr(flipud(semanticseg(flipud(fliplr(Irs)), net,
'ExecutionEnvironment','auto')));
PR = vote4_TAB(PR0,PR1,PR2,PR3,classNames);
else
PR = semanticseg(Irs, net, 'ExecutionEnvironment','auto');
end

% VARI vegetation gate (low veg => others)
if USE_VARI_GATE
VARI = vari_from_rgb_TAB(Irs);
```

```
lowVeg = VARI < VARI_THR;
PR(lowVeg & (PR=="weed" | PR=="rice")) = "others";
end

prIdx = cat2idx2D_TAB(PR, classNames); % 1..K

% Morphological area opening -> suppress tiny islands to "others"
if AREA_OPEN_PIX > 0
for kk=1:K
bw = (prIdx==kk);
bwClean = bwareaopen(bw, AREA_OPEN_PIX);
prIdx((prIdx==kk) & ~bwClean) = uint8(kOthers);
end
end

if MODEFILT_W>=3 && mod(MODEFILT_W,2)==1
prIdx = modefilt2_TAB(prIdx, MODEFILT_W);
end

% Restrict to labeled mask
gt = gtIdx(maskL);
pr = prIdx(maskL);

% Confusion matrix (GT rows x PR cols)
C = accumarray([gt, pr], 1, [K, K]);
confMat = confMat + C;

% Per-class TP/FP/FN/TN contributions
for kk=1:K
tp = C(kk,kk);
fp = sum(C(:,kk)) - tp;
fn = sum(C(kk,:)) - tp;
tn = sum(C(:)) - tp - fp - fn;
TP(kk)=TP(kk)+tp; FP(kk)=FP(kk)+fp; FN(kk)=FN(kk)+fn; TN(kk)=TN(kk)+tn;
classPixelsGT(kk) = classPixelsGT(kk) + sum(C(kk,:));
end

totalCorrect = totalCorrect + sum(pr==gt);
totalPixels = totalPixels + numel(gt);
if mod(i,5)==0 || i==N, fprintf(' * Processed %d/%d tiles…\n', i, N); end
end

% --- Per-class metrics ---
prec = TP ./ max(TP+FP, 1);
```

```
rec = TP ./ max(TP+FN, 1);
F1 = (2*prec.*rec) ./ max(prec+rec, eps);
IoU = TP ./ max(TP+FP+FN, 1);

% --- Aggregates ---
OA = totalCorrect / max(totalPixels,1);
microF1 = OA; % in single-label multiclass, Micro-F1 equals accuracy
macroF1 = mean(F1);
mIoU = mean(IoU);

% Frequency-weighted IoU
freq = classPixelsGT / max(sum(classPixelsGT),1);
FWIoU = sum(freq .* IoU);

% Cohen's kappa
po = OA;
pe = sum( (sum(confMat,1) .* sum(confMat,2)') ) / (sum(confMat(:))^2);
kappa = (po - pe) / max(1 - pe, eps);

% Multiclass MCC (Gorodkin 2004)
MCC = mcc_multiclass_TAB(confMat);

% --- Build integrated table ---
Section = {};
MetricClass = {};
Pcol = []; Rcol = []; F1col = []; IoUcol = []; Value = [];

% Overall block
addOverall = @(name,val) ( ...
[ { "Overall" }, { string(name) }, NaN, NaN, NaN, NaN, val ] );
rows = [ ...
addOverall("Overall Accuracy (OA)", OA); ...
addOverall("Micro-F1", microF1); ...
addOverall("Macro-F1", macroF1); ...
addOverall("Mean IoU (mIoU)", mIoU); ...
addOverall("Freq-Weighted IoU", FWIoU); ...
addOverall("Cohen''s \\kappa", kappa); ...
addOverall("Matthews Corr. (MCC)", MCC) ...
];

for rr = 1:size(rows,1)
Section{end+1,1} = rows{rr,1};
MetricClass{end+1,1} = rows{rr,2};
Pcol(end+1,1) = rows{rr,3};
```

```
Rcol(end+1,1) = rows{rr,4};
F1col(end+1,1) = rows{rr,5};
IoUcol(end+1,1) = rows{rr,6};
Value(end+1,1) = rows{rr,7};
end

% Per-class block
pcNames = ["Others","Rice","Weed"];
order = [kOthers, kRice, kWeed]; % ensure order
for j = 1:numel(order)
kk = order(j);
Section{end+1,1} = "Per-class";
MetricClass{end+1,1} = pcNames(j);
Pcol(end+1,1) = prec(kk);
Rcol(end+1,1) = rec(kk);
F1col(end+1,1) = F1(kk);
IoUcol(end+1,1) = IoU(kk);
Value(end+1,1) = NaN;
end

T = table( string(Section), string(MetricClass), ...
Pcol, Rcol, F1col, IoUcol, Value, ...
'VariableNames',
{'Section','Metric_Class','Precision','Recall','F1','IoU','Value'});

% --- Save CSV ---
writetable(T, outCsv);
fprintf('✓ Saved integrated metrics table (CSV) → %s\n', outCsv);

% --- Save LaTeX table (tabular) ---
fid = fopen(outTex,'w');
assert(fid>0, 'Cannot write LaTeX file: %s', outTex);
fprintf(fid, '%% Auto-generated by STEP 4F\n');
fprintf(fid, '\\begin{tabular}{l l r r r r r}\n');
fprintf(fid, '\\toprule\n');
fprintf(fid, 'Section & Metric / Class & Precision & Recall & F1 (Dice) & IoU
& Value \\\\\n');
fprintf(fid, '\\midrule\n');
for i=1:height(T)
s = char(T.Section(i));
mc = char(T.Metric_Class(i));
pr = T.Precision(i); rc = T.Recall(i); f1 = T.F1(i); iu = T.IoU(i); vv =
T.Value(i);
fprintf(fid, '%s & %s & %s & %s & %s & %s & %s \\\\\n', ...
```

```
s, mc, numfmt_TAB(pr), numfmt_TAB(rc), numfmt_TAB(f1), numfmt_TAB(iu),
numfmt_TAB(vv));
end
fprintf(fid, '\\bottomrule\n');
fprintf(fid, '\\end{tabular}\n');
fclose(fid);
fprintf('✓ Saved LaTeX table → %s\n', outTex);

%% ===== Helpers (unique suffix *_TAB) =====
function pairs = read_manifest_pairs_TAB(pth)
txt = string(splitlines(string(fileread(pth))));
txt = strip(txt); txt = txt(txt~="");
txt = replace(txt, """", ""); txt = txt(contains(txt,"|"));
if isempty(txt), pairs = strings(0,2); return; end
C = cellfun(@(s) strsplit(s,'|'), cellstr(txt), 'uni', 0);
C = C(cellfun(@(c) numel(c)==2, C));
pairs = string(vertcat(C{:}));
end

function VARI = vari_from_rgb_TAB(Iu8)
I = im2double(Iu8); R=I(:,:,1); G=I(:,:,2); B=I(:,:,3);
VARI = (G - R) ./ max(G + R - B, 1e-6);
end

function PR = vote4_TAB(A,B,C,D, classNames)
Ia = cat2idx2D_TAB(A, classNames);
Ib = cat2idx2D_TAB(B, classNames);
Ic = cat2idx2D_TAB(C, classNames);
Id = cat2idx2D_TAB(D, classNames);
stack = cat(3, Ia, Ib, Ic, Id);
PRidx = mode(stack, 3);
PR = idx2cat2D_TAB(PRidx, classNames);
end

function idx2D = cat2idx2D_TAB(C, classNames)
K = numel(classNames);
idx2D = zeros(size(C), 'uint8');
for k=1:K, idx2D(C==classNames(k)) = k; end
end

function C = idx2cat2D_TAB(idx, classNames)
C = categorical(zeros(size(idx)), 1:numel(classNames), cellstr(classNames));
for k=1:numel(classNames)
C(idx==k) = classNames(k);
```

```
end
end

function out = modefilt2_TAB(idx, w)
if ~(w>=3 && mod(w,2)==1), out=idx; return; end
pad=floor(w/2); idxp=padarray(idx,[pad pad],'replicate','both'); out=idx;
for r=1:size(idx,1)
for c=1:size(idx,2)
win=idxp(r:r+2*pad, c:c+2*pad); out(r,c)=mode(win(:));
end
end
end

function MCC = mcc_multiclass_TAB(C)
% Multiclass MCC from confusion matrix C (Gorodkin 2004)
C = double(C);
t = sum(C,2); % GT totals per class
p = sum(C,1)'; % Pred totals per class
c = sum(diag(C));
s = sum(C(:));
sum_pk_tk = sum(p .* t);
sum_pk2 = sum(p .^ 2);
sum_tk2 = sum(t .^ 2);
num = (c * s) - sum_pk_tk;
den = sqrt( (s^2 - sum_pk2) * (s^2 - sum_tk2) );
if den<=0, MCC = 0; else, MCC = num / den; end
end

function s = numfmt_TAB(x)
% Safe numeric -> char; NaN -> em dash
if ~isnumeric(x) || ~isscalar(x)
s = '—';
return;
end
if isnan(x)
s = '—';
else
s = sprintf('%.4f', x);
end
end
```

```
%%
```

```
%% =========================
% STEP 2.4 — Ground-truth summary for paper (counts, areas, visuals) [ROBUST
PARSE]
% =========================

clc;

% ---------- Paths ----------
workDir = fullfile("G:\D:\weedgraphnet\Nas Project\update imagery\Updated
clipped\Smallarea","small image matlab");
inLBL = fullfile(workDir,"..","label_small.tif"); % 0=unlabeled, 1=weed,
2=rice, 3=others
tilesDir = fullfile(workDir,"tiles");
lblDir = fullfile(tilesDir,"labels"); %#ok<NASGU>
listDir = fullfile(workDir,"lists");
trainList= fullfile(listDir,'train_manifest.txt');
valList = fullfile(listDir,'val_manifest.txt');

outDir = fullfile(workDir,"metrics","section_2_4");
if ~exist(outDir,"dir"), mkdir(outDir); end

% ---------- Assumptions for area conversion ----------
gsd_cm = 1.5; % Ground Sampling Distance ≈ 1.5 cm
px_m2 = (gsd_cm/100)^2; % m^2 per pixel

% ---------- Class meta ----------
classNames = ["Weed","Rice","Others"]; % paper names
classIDs = [1 2 3]; % 1=weed,2=rice,3=others

% ---------- Helper: safe manifest reader ----------
function pairs = readListSafe(p)
```

```
% Returns Nx2 string array [img,label]; skips malformed lines
pairs = strings(0,2);
if ~isfile(p), return; end

raw = string(splitlines(fileread(p)));
raw = strip(raw);
raw = raw(raw~="");
raw = replace(raw, """", ""); % strip quotes
raw = raw(contains(raw, "|")); % must contain delimiter

if isempty(raw), return; end

buf = strings(0,2);
for i = 1:numel(raw)
parts = split(raw(i), "|");
if numel(parts) ~= 2, continue; end
imgp = strtrim(parts(1));
lblp = strtrim(parts(2));
if imgp=="" || lblp=="", continue; end
buf(end+1, :) = [imgp, lblp]; %#ok<AGROW>
end
pairs = buf;
end

% ---------- 1) Full raster-level stats ----------
assert(isfile(inLBL), "Label raster not found: %s", inLBL);
Lfull = imread(inLBL);
if ndims(Lfull) > 2, Lfull = Lfull(:,:,1); end
Lfull = uint8(Lfull);

totPx = numel(Lfull);
unlabPx = nnz(Lfull==0);

pixCounts = zeros(1,3);
for k=1:3, pixCounts(k) = nnz(Lfull==classIDs(k)); end

labeledTotal = max(totPx - unlabPx,1);
pixFrac = pixCounts / labeledTotal; % fraction of labeled area
areas_m2 = pixCounts * px_m2;
areas_ha = areas_m2 / 1e4;

% ---------- 2) Train/Val manifest stats ----------
pairsTr = readListSafe(trainList);
pairsVa = readListSafe(valList);
```

```
fprintf('[2.4] Found TRAIN tiles: %d | VAL tiles: %d\n', size(pairsTr,1),
size(pairsVa,1));

sumPixTr = zeros(1,3); sumPixVa = zeros(1,3);
tileHasClassTr = zeros(1,3); tileHasClassVa = zeros(1,3);
countThresh = 0.01; % tile "has class" if >=1% pixels belong to that class

for i=1:size(pairsTr,1)
L = imread(pairsTr(i,2));
np = numel(L);
for k=1:3
c = nnz(L==classIDs(k));
sumPixTr(k) = sumPixTr(k) + c;
tileHasClassTr(k) = tileHasClassTr(k) + double(c/np >= countThresh);
end
end

for i=1:size(pairsVa,1)
L = imread(pairsVa(i,2));
np = numel(L);
for k=1:3
c = nnz(L==classIDs(k));
sumPixVa(k) = sumPixVa(k) + c;
tileHasClassVa(k) = tileHasClassVa(k) + double(c/np >= countThresh);
end
end

fracTr = sumPixTr / max(sum(sumPixTr),1);
fracVa = sumPixVa / max(sum(sumPixVa),1);

% ---------- 3) Connected-object counts (8-connectivity) ----------
objCounts = zeros(1,3);
for k=1:3
BW = (Lfull==classIDs(k));
CC = bwconncomp(BW,8);
objCounts(k) = CC.NumObjects;
end

% ---------- 4) Save tables ----------
T_full = table( ...
classNames.', pixCounts.', 100*pixFrac.', areas_m2.', areas_ha.',
objCounts.', ...
```

```
'VariableNames',
{'Class','PixelCount','PixelShare_pct','Area_m2','Area_ha','ConnectedObject
s'} ...
);

T_train = table( ...
classNames.', sumPixTr.', 100*fracTr.', tileHasClassTr.', ...
'VariableNames',
{'Class','Train_Pixels','Train_Share_pct','Train_Tiles_with_ge_1pct'} ...
);

T_val = table( ...
classNames.', sumPixVa.', 100*fracVa.', tileHasClassVa.', ...
'VariableNames',
{'Class','Val_Pixels','Val_Share_pct','Val_Tiles_with_ge_1pct'} ...
);

writetable(T_full , fullfile(outDir,'labels_full_area_summary.csv'));
writetable(T_train, fullfile(outDir,'labels_train_distribution.csv'));
writetable(T_val , fullfile(outDir,'labels_val_distribution.csv'));

% ---------- 5) TXT summary ----------
fid = fopen(fullfile(outDir,'section_2_4_summary.txt'),'w');
fprintf(fid, "Ground reference summary (Section 2.4)\n");
fprintf(fid, "Label raster: %s\n", inLBL);
fprintf(fid, "Total pixels (incl. unlabeled): %d\n", totPx);
fprintf(fid, "Unlabeled pixels: %d (%.2f %%)\n\n", unlabPx,
100*unlabPx/max(totPx,1));
for k=1:3
fprintf(fid, "%s: pixels=%d | share=%.2f %% | area=%.2f m^2 (%.4f ha) | connected
objects=%d\n", ...
classNames(k), pixCounts(k), 100*pixFrac(k), areas_m2(k), areas_ha(k),
objCounts(k));
end
fprintf(fid,"\nTRAIN tiles: %d | VAL tiles: %d\n", size(pairsTr,1),
size(pairsVa,1));
for k=1:3
fprintf(fid, "TRAIN %s: pixels=%d (%.2f %%) | tiles with >=1%%=%d\n", ...
classNames(k), sumPixTr(k), 100*fracTr(k), tileHasClassTr(k));
end
for k=1:3
fprintf(fid, "VAL %s: pixels=%d (%.2f %%) | tiles with >=1%%=%d\n", ...
classNames(k), sumPixVa(k), 100*fracVa(k), tileHasClassVa(k));
end
```

```
fclose(fid);

% ---------- 6) Figures ----------
% Pie labels like "Weed (xx.x%)"
labelsPie = cellstr(classNames + " (" + string(compose('%.1f',100*pixFrac)) +
"%)");
fig = figure('Color','w','Position',[100 100 620 480]);
pie(pixFrac, labelsPie);
colormap([255 215 0; 220 20 60; 150 150 150]/255); % Weed,Rice,Others
title('Class composition of labeled ground reference (full raster)');
saveas(fig, fullfile(outDir,'fig_2_4a_pie_full_labels.png')); close(fig);

% TRAIN vs VAL stacked shares
fig = figure('Color','w','Position',[100 100 820 420]);
bar([fracTr; fracVa], 'stacked');
colormap([255 215 0; 220 20 60; 150 150 150]/255);
xticklabels({'TRAIN','VAL'}); ylim([0 1]);
ylabel('Fraction of labeled pixels'); grid on; box on;
legend(classNames,'Location','eastoutside');
title('TRAIN vs VAL labeled pixel composition');
saveas(fig, fullfile(outDir,'fig_2_4b_stacked_train_val.png')); close(fig);

% Connected-object counts bar
fig = figure('Color','w','Position',[100 100 700 420]);
bar(objCounts, 'FaceColor',[0.6 0.6 0.6]); box on; grid on;
xticks(1:3); xticklabels(classNames);
ylabel('Connected objects (approx. "polygons")');
title('Connected-object counts from raster labels (8-neighborhood)');
saveas(fig, fullfile(outDir,'fig_2_4c_objects_bar.png')); close(fig);

% ---------- 7) Echo key numbers ----------
fprintf('\n=== Section 2.4 key numbers ===\n');
for k=1:3
fprintf('%-6s Pix=%8d | Share=%6.2f %% | Area=%.1f m^2 (%.4f ha) |
Objects≈%d\n', ...
classNames(k), pixCounts(k), 100*pixFrac(k), areas_m2(k), areas_ha(k),
objCounts(k));
end
fprintf('TRAIN tiles=%d | VAL tiles=%d\n\n', size(pairsTr,1),
size(pairsVa,1));



%%
```

```
%% FINAL FULL-SIZE SEGMENTATION EXPORT (PNG)
clc; rng('default'); rng(42);

% -------- Paths (match your project) --------
workDir = fullfile("G:\D:\weedgraphnet\Nas Project\update imagery\Updated
clipped\Smallarea","small image matlab");
inRGB = fullfile(workDir,"..","small_valid_8bit.tif"); % full patch RGB (8-bit)
mdlDir = fullfile(workDir,"train"); % contains model_unet_*.mat
outDir = fullfile(workDir,"metrics","full_export");
if ~exist(outDir,'dir'), mkdir(outDir); end

% -------- Load latest model --------
mats = dir(fullfile(mdlDir,"model_unet_*.mat"));
assert(~isempty(mats),'No trained model found in %s', mdlDir);
[~,ix] = sort([mats.datenum],'descend');
S = load(fullfile(mdlDir,mats(ix(1)).name));
assert(isfield(S,'net'),'Model file missing variable ''net'': %s',
mats(ix(1)).name);
net = S.net;

% Class meta (fallbacks if absent in .mat)
classNames = ["others","rice","weed"];
labelIDs = [3 2 1]; % mapping for PNG: 1=weed, 2=rice, 3=others
inSize = [512 512 3];

% -------- Read full patch + band handling --------
infoI = imfinfo(inRGB);
H = infoI(1).Height; W = infoI(1).Width;
I0 = imread(inRGB); % could be 3 or 4 bands; we enforce RGB first 3
if ndims(I0)==2, I0 = repmat(I0,1,1,3); end
if size(I0,3) >= 3, I0 = I0(:,:,1:3); else
I3 = zeros(H,W,3,'like',I0); I3(:,:,1:size(I0,3))=I0; I0 = I3;
end
I0 = im2uint8(I0);

% -------- Tile grid (stride=512, pad edge tiles by replication) --------
ts = inSize(1:2); st = ts;
rows = 1:st(1):H; cols = 1:st(2):W;
if rows(end) ~= H - ts(1) + 1, rows = [rows, max(1, H-ts(1)+1)]; end
if cols(end) ~= W - ts(2) + 1, cols = [cols, max(1, W-ts(2)+1)]; end

% -------- Output buffers --------
```

```
finalIdx = zeros(H,W,'uint8'); % will hold {1=weed,2=rice,3=others}

% -------- Parameters for refinement --------
useTTA = true; % flip-based TTA
VARI_thr = 0.05; % vegetation gate (RGB-only)
minIslandPx = 100; % remove tiny islands per class (≈2.25 cm² at 1.5 cm GSD)

% -------- Process tiles --------
fprintf('Full inference on %dx%d…\n',H,W);
tStart = tic;
for r = rows
r1 = r; r2 = min(H, r1+ts(1)-1);
for c = cols
c1 = c; c2 = min(W, c1+ts(2)-1);

% Crop & pad to 512×512
Ic = I0(r1:r2, c1:c2, :);
if any(size(Ic,1:2) ~= ts)
Ic = padarray(Ic, ts - size(Ic,1:2), 'replicate', 'post');
end

% --- Predict with TTA (majority vote over flips) ---
PR = predict_categorical(Ic, net, classNames, useTTA);

% --- Vegetation gating (VARI) to suppress false weed/rice on non-veg ---
PR = apply_VARI_gate(Ic, PR, classNames, VARI_thr);

% --- Morphology: remove tiny islands, per class ---
PR = morphology_clean(PR, classNames, minIslandPx);

% Crop back to original subwindow size (if padded)
PRc = PR(1:(r2-r1+1), 1:(c2-c1+1));

% Convert categorical to index map {1=weed,2=rice,3=others}
idxTile = cat_to_ids(PRc, classNames, labelIDs);

% Write into full canvas
finalIdx(r1:r2, c1:c2) = idxTile;
end
end
fprintf('Done in %.1f s.\n', toc(tStart));

% -------- Save outputs (PNG) --------
% Color LUT: weed(yellow), rice(red), others(gray)
```

```
cmap = uint8([255 215 0; 220 20 60; 150 150 150]); % rows correspond to
{weed,rice,others}

finalRGB = ids_to_rgb(finalIdx, cmap); % colorized
overlayRGB = overlay(I0, finalRGB, 0.45); % blend for figure

imwrite(finalIdx, fullfile(outDir,'final_mask_full.png')); % indexed class map
imwrite(finalRGB, fullfile(outDir,'final_color_full.png')); % color mask
imwrite(overlayRGB, fullfile(outDir,'final_overlay_full.png')); % overlay

fprintf('✓ Saved:\n %s\n %s\n %s\n', ...
fullfile(outDir,'final_mask_full.png'), ...
fullfile(outDir,'final_color_full.png'), ...
fullfile(outDir,'final_overlay_full.png'));

%% ==================== Local helpers ====================

function PR = predict_categorical(I, net, classNames, useTTA)
% Predict categorical map with optional flip-TTA (majority vote).
if ~useTTA
PR = semanticseg(I, net, 'ExecutionEnvironment','auto');
return;
end
preds = cell(4,1);

% 1) original
preds{1} = semanticseg(I, net, 'ExecutionEnvironment','auto');

% 2) fliplr
I2 = fliplr(I);
P2 = semanticseg(I2, net, 'ExecutionEnvironment','auto');
preds{2} = fliplr(P2);

% 3) flipud
I3 = flipud(I);
P3 = semanticseg(I3, net, 'ExecutionEnvironment','auto');
preds{3} = flipud(P3);

% 4) flipud+fliplr
I4 = flipud(fliplr(I));
P4 = semanticseg(I4, net, 'ExecutionEnvironment','auto');
preds{4} = fliplr(flipud(P4));

% Majority vote per pixel
```

```
% Convert to numeric labels 1..K based on classNames order
K = numel(classNames);
V = zeros([size(I,1), size(I,2), K], 'uint16'); % votes per class
for t = 1:numel(preds)
tmp = preds{t};
for k=1:K
V(:,:,k) = V(:,:,k) + uint16(tmp == classNames(k));
end
end
[~,win] = max(V, [], 3);
% map back to categorical
PR = categorical(win, 1:K, cellstr(classNames));
end

function PR = apply_VARI_gate(I, PR, classNames, thr)
% Suppress weed/rice where greenness is too low (non-vegetation → Others).
I = im2double(I);
R = I(:,:,1); G = I(:,:,2); B = I(:,:,3);
VARI = (G - R) ./ max(G + R - B, eps);
nonveg = VARI < thr;

isWeed = (PR == classNames(end)); % by our order ["others","rice","weed"]
isRice = (PR == classNames(2));
PR(nonveg & (isWeed | isRice)) = classNames(1); % → Others
end

function PR = morphology_clean(PR, classNames, minArea)
% Remove tiny islands for each class independently; fill tiny holes in rice.
% Weed cleanup
BWw = (PR==classNames(3));
BWw = bwareaopen(BWw, minArea, 8);
% Rice cleanup
BWr = (PR==classNames(2));
BWr = bwareaopen(BWr, minArea, 8);
BWr = imfill(BWr,'holes');
% Others cleanup
BWo = (PR==classNames(1));
BWo = bwareaopen(BWo, minArea, 8);

% Recompose respecting exclusivity (priority: rice > weed > others)
% (You can change priority if needed.)
PR(:) = classNames(1);
PR(BWw) = classNames(3);
PR(BWr) = classNames(2);
```

```
end

function idx = cat_to_ids(C, classNames, labelIDs)
% categorical C (others/rice/weed) → uint8 {1=weed,2=rice,3=others}
idx = zeros(size(C),'uint8');
for k = 1:numel(classNames)
idx(C==classNames(k)) = uint8(labelIDs(k));
end
end

function RGB = ids_to_rgb(idx, cmap)
% idx {1=weed,2=rice,3=others} → uint8 RGB via LUT rows [weed;rice;others]
LUT = zeros(256,3,'uint8');
LUT(1,:) = cmap(1,:); % 1->weed
LUT(2,:) = cmap(2,:); % 2->rice
LUT(3,:) = cmap(3,:); % 3->others
s = size(idx);
RGB = cat(3, reshape(LUT(idx,1),s), reshape(LUT(idx,2),s),
reshape(LUT(idx,3),s));
end

function J = overlay(I, M, alpha)
% Simple alpha blend (uint8 in, uint8 out)
I = im2double(I); M = im2double(M);
J = im2uint8( (1-alpha)*I + alpha*M );
end
```